

WiseBuy AI: An Intelligent Real-Time Shopping Assistant

A Conversational AI System for Multi-Platform Product Recommendation

Submitted by

Afreen Sultana

Mohammed Safwan Adil

Thiago Guiotti de Freitas

Under the Guidance of: Dr. Ziad Al Sharif

Computer Science Master's Project

Department of Computer Science

Lewis University

Romeoville, Illinois

Date- 05 May 2026

Submitted partial fulfillment of the requirements for the Master of Science in Computer Science

ABSTRACT

Online shopping presents a growing challenge: consumers must manually compare products across multiple platforms with inconsistent prices, unreliable reviews, and no personalized guidance. WiseBuy AI addresses this by providing an intelligent conversational shopping assistant that searches Amazon and Walmart in real time and delivers ranked, sentiment-enriched product recommendations through a natural-language chat interface. The system integrates a LLaMA 3.3-70b large language model served through Groq Cloud for multi-turn conversation, a locally deployed DistilBERT model for CPU-based sentiment scoring, and concurrent real-time API calls to both marketplaces using Python's ThreadPoolExecutor.

Products are ranked using a composite formula combining star ratings, review volume, and sentiment scores, then displayed across Picks and Trending tabs. Additional features include image-based product search using a LLaMA 4 Scout vision model, user authentication with PBKDF2-SHA256 password hashing, persistent chat history, clipboard paste image input, discount badge display, and dark and light theme support. All 22 functional test cases passed during evaluation.

The prototype demonstrates that combining conversational AI with live marketplace data and local sentiment analysis produces a more accurate and user-friendly shopping experience than traditional popularity-based recommendation systems.

Keywords: Conversational AI, Large Language Model, Sentiment Analysis, DistilBERT, Product Recommendation, Flask, Real-Time API, E-Commerce, LLaMA

Table of Content	Page no's
I. Introduction	1
II. Literature Survey	2
III. System Requirements Specification	6
3.1 System Overview	6
3.2 Functional Requirements	9
3.3 Non-Functional Requirements	10
3.4 System Requirements	10
3.5 Use Case Model	12
3.5.1 Activity Diagram	14
IV. System Design	16
4.1 Architectural Design	16
4.2 Development Model	18
V. Implementation Cycle	19
5.1 Technology Stack	19
5.1.1 Logical View	19
5.1.2 Physical View	21
5.1.3 Development View	22
5.1.4 Process View	24
5.2 Database Design	25
5.3 security Design	27
5.4 Feature by Feature Walkthrough	27
VI. Tests and Results	29
6.1 Demo Scenarios	30
6.1 Limitations	30
VII Conclusion	31
VIII. Future Enhancements	33
References	35

List of Tables	Page no's
3.1 System Interface	6
3.2.1 User Account Management	7
3.2.2 Product Search	7
3.2.3 Sentiment-Analysis and Recommendation Ranking	8
3.2.4 Conversational Chatbot	8
3.2.5 Image-based Product Search	9
3.2.6 Logical Database Requirements	9
3.3.1 Performance Requirements	9
3.3.2 Safety and security Requirements	10
5.2.1 User	26
5.2.2 Chat history	26
5.2.3 Password Reset Token	26
6.1 Test Environments	29
6.2 Test Results	29-30

List of Figures	Pg no's
3.5.1 Use Case Diagram	14
3.5.2 Activity Diagram	15
4.1 system Architecture	17
4.2 Implemental Development Cycle	18
5.1.1 Component Diagram	20
5.1.2 Deployment Diagram	22
5.1.3 Package Diagram	23
5.1.4 Sequence Diagram	25
5.3.1 User Interface	27

I. INTRODUCTION

Online shopping has made product access easier than ever, but it has also made choosing harder. Walk into any store and a salesperson can ask what you need, understand your budget, and point you to something that fits -that kind of guided experience barely exists online. What exists instead is an overwhelming flood of search results, inconsistent reviews, and prices that vary between Amazon and Walmart for the exact same item. Users end up spending more time researching than they do shopping.

WiseBuy AI was built to fix that. Instead of making users compare products themselves, the system has a conversation with them -asking what they need, what their budget is, and what features matter -then searches Amazon and Walmart simultaneously and ranks the results using both star ratings and a sentiment score from a locally run AI model. The final recommendation comes back as a plain-English explanation alongside ranked product cards. It works like a knowledgeable salesperson: it listens first, then searches.

The system connects to live Amazon and Walmart APIs, so every result reflects real current pricing and availability. A conversational AI named Kart handles the dialogue using LLaMA 3.3-70b, a DistilBERT model running on the server scores review sentiment locally without any external API call, and a LLaMA 4 Scout vision model identifies products from uploaded or pasted photos. Everything runs inside a single Python Flask process -no separate servers, no build pipeline, no complex infrastructure to manage.

The rest of this paper is organized as follows. Section II reviews the related literature that informed this work. Section III presents the Software Requirements Specification covering functional and non-functional requirements. Section IV covers the Software Design Document including system architecture, database design, and security. Section V describes the implementation of each feature. Section VI presents testing results and demo scenarios. Section VII discusses findings, limitations, and future directions. Section VIII concludes the paper.

Definitions

Conversational AI: A type of artificial intelligence that enables computers to understand and respond to human language in a natural, dialogue-based way. Instead of typing commands or filling out forms, users simply describe what they want in their own words, and the system responds like a person would. In WiseBuy AI, conversational AI powers the Kart assistant that asks follow-up

questions and guides the user toward a product recommendation through back-and-forth conversation.

Large Language Model (LLM): A neural network trained on massive amounts of text data that can understand context, generate human-like responses, summarize information, and follow complex instructions. LLMs learn language patterns from billions of sentences, which allows them to handle open-ended questions and maintain conversational context across multiple turns. WiseBuy AI uses LLaMA 3.3-70b served through Groq Cloud as its primary language model.

Sentiment Analysis: The process of using AI to determine whether a piece of text expresses a positive, negative, or neutral opinion. In the context of product reviews, sentiment analysis goes beyond star ratings by reading the actual language of a review and judging how satisfied the reviewer really was. WiseBuy AI applies sentiment analysis to product review summaries and uses the result as part of the final ranking score.

DistilBERT: A lightweight version of the BERT transformer model developed by Hugging Face. It is 40% smaller and 60% faster than the original BERT while retaining 97% of its language understanding capability. DistilBERT was created using a technique called knowledge distillation, where a smaller model is trained to mimic the behavior of a larger one. In WiseBuy AI, DistilBERT runs locally on CPU server and scores each product's review text with a sentiment value between 0.0 and 1.0.

Product Recommendation: The process of suggesting products to a user based on their specific needs, preferences, and budget rather than just showing what is most popular or most purchased. A good recommendation system considers multiple factors price fit, quality signals, review sentiment, and user intent and presents results in a ranked order that prioritizes the best match for that user.

Flask A lightweight web framework written in Python that handles routing, request processing, and server-side logic. Flask is described as a micro-framework because it provides the essential tools to build a web application without forcing a specific project structure or requiring many dependencies. WiseBuy AI uses Flask as its sole backend framework all routes, authentication, product search, streaming, and database access are managed through one Flask application.

Real-Time API An Application Programming Interface that returns live, current data at the moment of the request rather than from a cached or pre-stored dataset. Real-time APIs reflect the actual state of a system right now - current prices, current availability, current inventory. WiseBuy AI

uses real-time APIs from RapidAPI (Amazon) and OpenWebNinja (Walmart) to ensure that every product result shown to the user reflects today's actual pricing and availability, not data that was collected days or weeks ago.

E-Commerce Electronic commerce - the buying and selling of products and services over the internet. E-commerce platforms like Amazon and Walmart allow consumers to browse millions of products, compare prices, read reviews, and complete purchases entirely online. The growth of e-commerce has created both opportunity and challenge: more products are accessible than ever but comparing them across multiple platforms is time-consuming and often overwhelming for consumers.

LLaMA Large Language Model Meta AI - a family of open-source large language models developed and released by Meta AI. LLaMA models range from 7 billion to 70 billion parameters and are available in both base and instruction-tuned chat variants. The chat versions were fine-tuned using Reinforcement Learning from Human Feedback (RLHF), making them capable of following complex multi-turn instructions and maintaining conversational context. WiseBuy AI uses llama-3.3-70b-versatile for conversation and meta-llama/llama-4-scout-17b for image-based product Identification

II. LITERATURE SURVEY

Background

The e-commerce industry reached \$6.8 trillion in global sales in 2025, with over 28 million active online stores. Despite that scale -or maybe because of it -most shoppers still struggle to make confident purchase decisions. The systems that exist today are largely popularity driven. They show you what other people bought, not necessarily what fits your situation. The AI shopping assistant market, valued at \$4.67 billion in 2025, is growing toward \$84.6 billion by 2034, which tells you how large the gap between what exists and what users really needs.

➤ **Mullagura (2023) -Traditional Sentiment-Based Recommendation**

Yuktha Mullagura's thesis from Blekinge Institute of Technology approached the product recommendation problem by applying sentiment analysis directly to Amazon review data. The dataset, pulled from the UCI Machine Learning Repository, contained 252,000 reviews. The methodology was straightforward: preprocessed the text (stop word removal, Porter stemming),

apply lexical analysis to extract sentiment polarity, then classify using Support Vector Machine and Naive Bayes algorithms.

The results were meaningful -SVM reached 95.03% accuracy compared to Naive Bayes at 85.08%, and the system could meaningfully display positive versus negative review percentages for each product. For 2023, this was solid work.

But the limitations are clear when you look at what the system cannot do. It only covered Amazon. It had no conversational interface, no way to understand what a specific user wanted, and no budget awareness. The output was a sentiment percentage, not an actionable recommendation. It told you a product was 78% positively reviewed -it could not tell you whether that product was right for you given your needs and budget.

WiseBuy AI addresses these gaps directly. Rather than running sentiment on raw review text from a stored dataset, we generate review blurbs through the LLM and score those with DistilBERT, integrating the sentiment score into a composite ranking alongside star ratings and review volume.

➤ **Tangarajan et al. (2025) -RAG-Based Conversational E-Commerce QA**

The Flipkart paper represents a more modern approach. Tangarajan and colleagues built a Retrieval-Augmented Generation (RAG) framework for product answering that served over 5 million monthly users in production. The system architecture was sophisticated: a fine-tuned LLaMA 3-8B model for query contextualization, a BERT-based intent classifier, bi-encoder retrieval, and structured LLM generation. Precision ranged from 90.3% to 98.7% across intent categories, and hallucination rates stayed below 7%.

What is notable about this work is how seriously it took conversational context. The system-maintained dialogue history and adapted to follow-up questions -which is much closer to how people shop. It did not just answer one question; it understood a thread of questions.

The limitation is that it was still a single-platform system. Flipkart only. A user asking, "is this cheaper on Amazon?" gets no answer. There is also no cross-platform ranking -you cannot know if the product being recommended is the best deal available that day across multiple retailers.

WiseBuy AI takes the conversational approach further by running real-time searches across both Amazon and Walmart simultaneously, so the recommendation is not just contextually aware but also currently accurate.

➤ **Sanh et al. (2019) -DistilBERT**

When we needed a sentiment model that could run locally on a CPU without requiring a GPU server or costly external API calls, DistilBERT was the obvious choice. Sanh and colleagues at Hugging Face introduced DistilBERT as a distilled version of BERT -40% smaller, 60% faster, and retaining 97% of BERT's language understanding capability. The distillation process used knowledge transfer from the larger BERT teacher model during training, producing a six-layer transformer that could handle real NLP tasks at a fraction of the compute cost.

In WiseBuy AI, the DistilBERT model runs in `scraper/sentiment_model/` and scores product review blurbs through a single batch forward pass. Each product receives a sentiment score between 0.0 and 1.0, which feeds directly into the final composite ranking formula. The efficiency Sanh et al. described -CPU-only inference without meaningful quality loss -is exactly what made local deployment feasible for this project.

➤ **Touvron et al. (2023) -LLaMA 2**

The conversational intelligence behind WiseBuy AI runs on llama-3.3-70b-versatile, a third-generation model from Meta's LLaMA family. Understanding where that capability comes from requires looking at LLaMA 2, the foundational work Touvron and colleagues published in 2023. LLaMA 2 scaled from 7 billion to 70 billion parameters and introduced instruction-tuned chat variants trained using Reinforcement Learning from Human Feedback (RLHF). RLHF training is what made these models useful for multi-turn conversation - they learned to follow complex instructions, maintain context across several turns, and generate responses that feel natural rather than mechanical.

The 70-billion-parameter chat model demonstrated competitive performance with GPT-3.5 on multiple benchmarks while remaining openly accessible. That lineage matters for WiseBuy AI. When Kart maintains a rolling 20-message conversation history and asks targeted clarifying questions about budget and features before searching -that capability traces directly to the training methodology Touvron et al. established in LLaMA 2 and carried forward into subsequent generations.

III. SOFTWARE REQUIREMENTS SPECIFICATION

3.1 System Overview

WiseBuy AI is a browser-based conversational shopping assistant built to solve a problem that sounds simple but turns out to be genuinely difficult: helping someone buy the right product across two major retailers without making them do the comparison work themselves. The system connects to live Amazon and Walmart APIs, scores results using a local sentiment model, and communicates with the user through a natural-language chatbot interface -all in a single Python process that can be started with one command.

The system boundary is clean. Everything runs through a Flask server: authentication, conversation management, product search orchestration, sentiment scoring, and streaming responses. The user's browser loads a single HTML file and maintains a persistent SSE connection for real-time token delivery. No separate API server, no build pipeline, no microservices -just one app.py doing everything.

Interface	Type		Protocol / Format	Purpose
Amazon Product Search API	External API	REST	HTTPS / JSON (RapidAPI host)	Retrieve real-time Amazon product listings, prices, ratings, and review counts.
Walmart Product Search API	External API	REST	HTTPS / JSON (OpenWebNinja)	Retrieve real-time Walmart product listings and pricing.
Groq Cloud Chat Completion	External API	LLM	HTTPS / JSON	Generate conversational chatbot responses using LLaMA 3.3-70b-versatile.
Groq Cloud Vision	External API	LLM	HTTPS / JSON (Base64 image)	Identify product search terms from a user-uploaded image using LLaMA 4 Scout.
DistilBERT Sentiment Model	Local ML model		PyTorch / HuggingFace Transformers	Score product review blurbs for positive sentiment (0.0–1.0).
SQLite Database	Embedded RDBMS	File I/O (SQLite3 protocol)		Persist user accounts, chat history, and password reset tokens.

Table 3.1 System Interfaces

3.2 Functional Requirements

The following functional requirements define the behavior the system must exhibit. Priority levels: H = High, M = Medium, L = Low.

ID	Requirement	Priority
FR-01	The system shall allow a new user to register by providing a unique username, a valid email address, and a password of at least 8 characters.	H
FR-02	The system shall hash passwords using PBKDF2-HMAC-SHA256 with a minimum of 260,000 iterations and a cryptographically random 32-byte salt before storing them in the database.	H
FR-03	The system shall send a verification email to the user's registered address upon registration and require email confirmation before enabling full account access.	H
FR-04	The system shall allow a registered user to log in using their email address or username and their password.	H
FR-05	The system shall create a server-side session on successful login, identified by an HttpOnly, SameSite=Lax cookie.	H
FR-06	The system shall allow a logged-in user to log out, invalidating the server-side session.	H
FR-07	The system shall provide a password reset flow: (a) user requests a reset by email, (b) server generates a cryptographically secure token valid for 20 minutes, (c) server sends the token link by email, (d) user submits a new password using the token, (e) server invalidates the token and updates the password hash.	H

Table 3.2.1 *User Account Management*

ID	Requirement	Priority
FR-08	The system shall accept a free-text product search query from the user, filter out common stop words (a, an, and, are, as, at, be, best, buy, by, for, etc.), and forward the cleaned query to the external product APIs.	H
FR-09	The system shall query the Amazon Product Search API and the Walmart Product Search API concurrently using a thread pool (ThreadPoolExecutor) to minimize total search latency.	H
FR-10	The system shall support optional budget parameters: when a budget is provided and exceeds \$30, the API call shall include a max_price filter equal to the budget; when the budget exceeds \$80, a min_price filter equal to 35% of the budget shall also be applied.	H
FR-11	The system shall normalize all price values to USD-formatted strings (e.g., \$1,234.56). Products with no parseable price shall display "N/A".	M
FR-12	The system shall return a maximum of 12 product results (RECOMMENDATION_COUNT = 12) per search, ranked by composite score.	H
FR-13	When the Amazon API returns an error or quota-exceeded message, the system shall fall back to a predefined set of 8 mock product entries rather than returning an error to the user.	M
FR-14	When the Walmart API is unavailable or returns a non-200 response, the system shall silently skip Walmart results and return only Amazon results.	M

Table 3.2.2 *Product Search*

ID	Requirement	Priority
FR-15	The system shall score each product's review blurb text using the local DistilBERT sentiment model, producing a POSITIVE confidence score in [0.0, 1.0]. If the model is unavailable, the system shall substitute a neutral score of 0.5.	H
FR-16	The system shall support batch inference: multiple review texts shall be scored in a single model forward pass using padded/truncated inputs (max 128 tokens).	M
FR-17	The system shall compute a composite product score using the formula: $\text{score} = (\text{star_rating} \times 100) + (\text{num_reviews} \times 0.01) + (\text{sentiment_score} \times 5)$, and rank results in descending order of this score.	H
FR-18	The DistilBERT model shall be loaded lazily on first use (not at server startup) and cached for the lifetime of the process.	M

Table 3.2.3 *Sentiment Analysis and Recommendation Ranking*

ID	Requirement	Priority
FR-19	The system shall expose a conversational chat interface powered by the LLaMA 3.3-70b-versatile model served via Groq Cloud. The chatbot persona is named "Kart."	H
FR-20	The chatbot shall operate according to a structured system prompt that instructs it to collect: (1) specific product type, (2) budget, and (3) optional feature preferences before issuing a search.	H
FR-21	The chatbot shall embed a [SEARCH: <query>] directive in its response to trigger a product search. The application layer shall detect this directive via regex, execute the search, and inject results back into the conversation as a user turn.	H
FR-22	The chatbot shall stream its response tokens to the browser using Server-Sent Events (SSE) to provide a real-time typing experience.	H
FR-23	The conversation context window shall be capped at the most recent 20 messages to prevent context overflow.	M
FR-24	The system shall persist each conversation turn (role, content, timestamp) to the SQLite chat_history table for authenticated users.	H
FR-25	The system shall allow authenticated users to retrieve their chat history for a given session ID via a REST API endpoint.	M

Table 3.2.4 *Conversational Chatbot (Kart)*

ID	Requirement	Priority
FR-26	The system shall accept image uploads of type image/jpeg, image/png, image/webp, or image/gif with a maximum size of 3 MB (3,145,728 bytes).	H
FR-27	The system shall encodes the uploaded image as a Base64 data URI and submit it to the Groq vision model (meta-llama/llama-4-scout-17b-16e-instruct) to extract a product search query.	H
FR-28	The system shall limit the number of image-triggered product search queries per conversation session to a maximum of 5 (MAX_IMAGE_SEARCH_QUERIES = 5).	M
FR-29	If the image type is not in the allowed set, the system shall reject the upload with an appropriate error message and not forward it to the vision model.	H

Table 3.2.5 *Image-Based Product Search*

The system uses an SQLite relational database with the following logical schema:

Table	Key Columns	Purpose
users	id (PK), username (UNIQUE), email (UNIQUE), password_hash, salt, email_verified, created_at	Stores registered user accounts with hashed credentials.
chat_history	id (PK), user_id (FK → users), session_id, role, content, timestamp	Persists chat messages per user per session for history retrieval.
password_reset_tokens	id (PK), user_id (FK → users), token_hash, expires_at, used	Stores time-limited password reset tokens (expire after 20 minutes).

Table 3.2.6 *Logical Database Requirements*

3.3 Non-Functional Requirements

ID	Requirement	Priority
NFR-01	The system shall return product search results to the browser within 8 seconds under normal network conditions, including concurrent Amazon and Walmart API calls.	H
NFR-02	The DistilBERT sentiment model complete batch inference on up to 12 product texts in under 5 seconds on a typical development CPU.	M
NFR-03	The first streamed chat token from the Groq LLM shall appear in the browser within 3 seconds of the user submitting a message.	M
NFR-04	The Flask server shall support at least 10 concurrent users without significant performance degradation on a standard single-core server instance.	M
NFR-05	Image upload and vision model processing (image-to-search-query) shall be completed within 15 seconds.	M
NFR-06	The user authentication flow (login, session creation) shall be completed in under 2 seconds under normal server load.	H

Table 3.3.1 *Performance Requirements*

ID	Requirement	Priority
NFR-07	All passwords must be stored as PBKDF2-HMAC-SHA256 hashes with a minimum of 260,000 iterations and a unique cryptographic salt per user. Plaintext passwords must never be stored or logged.	H
NFR-08	Session cookies must be set with HttpOnly=True and SameSite=Lax to mitigate XSS and CSRF attacks. The Secure flag must be set to True in any HTTPS production deployment.	H
NFR-09	Password reset tokens must be generated using a cryptographically secure random source, stored as hashes (not plain text), and expire after 20 minutes. Each token may be used at most once.	H
NFR-10	Uploaded images must be validated for MIME type (image/jpeg, image/png, image/webp, image/gif) and size (≤ 3 MB) before being processed. Invalid uploads must be rejected with an HTTP 400 error.	H
NFR-11	All API keys and secrets (FLASK_SECRET_KEY, GROQ_API_KEY, RAPIDAPI_KEY, OPENWEBNINJA_KEY, SMTP credentials) must be supplied via environment variables and must not appear in version-controlled source code.	H
NFR-12	The system must not expose internal Python stack traces or database error details to the browser client in production mode. Error responses must use generic user-facing messages.	H
NFR-13	The system must enforce a minimum password length of 8 characters at registration and password reset time.	H

Table 3.3.2 Safety and Security Requirements

3.4 System Requirements

To run the WiseBuy AI project successfully, the system must meet certain hardware and software requirements. Since the project uses a Flask backend, SQLite database, AI integration, external product APIs, and a local sentiment analysis model, the system should support Python-based web development and basic machine learning dependencies.

Hardware Requirements

The minimum hardware requirements are:

Component	Minimum Requirement	Recommended Requirement
Processor	Intel i3 or equivalent	Intel i5/i7 or equivalent
RAM	4 GB	8 GB or higher
Storage	2 GB free space	5 GB or higher
Internet	Required	Stable broadband connection

A table internet connection is important because the system communicates with external services such as Groq, Amazon product API, and Walmart product API.

Software Requirements

The software requirements for running the project are:

Software	Requirement
Operating System	Windows, Linux, or macOS
Python	Python 3.10 or above
Web Framework	Flask
Database	SQLite
Browser	Chrome, Edge, Firefox, or any modern browser
Code Editor	VS Code or any Python-supported IDE
Package Manager	pip

Python Libraries Required

The project requires the following Python packages:

- flask
- groq
- python-dotenv
- requests
- torch
- transformers
- safetensors

These libraries are used for web application development, API communication, environment variable management, AI integration, and sentiment analysis.

Environment Variables Required

The project also requires API keys to enable full functionality:

Variable	Purpose
GROQ_API_KEY	Used for AI chatbot and image understanding
RAPIDAPI_KEY	Used to fetch Amazon product data
OPENWEBNINJA_KEY	Used to fetch Walmart product data
FLASK_SECRET_KEY	Used to secure Flask sessions

If these keys are not provided, some features may not work completely. For example, the chatbot needs the Groq API key, while real-time product search requires marketplace API keys.

Overall, WiseBuy AI can run on a standard laptop or desktop system with Python installed. The recommended setup is a system with at least 8 GB RAM, Python 3.10 or above, a modern web browser, and a stable internet connection for accessing AI and product search services.

3.5 Use Case Model

The following use cases describe the primary interactions between users and WiseBuy AI.

UC-01	Register Account
Actor	Anonymous User
Description	User submits registration form. System validates inputs, hashes password, stores user, and sends verification email.
Precondition	User has not registered.
Postcondition	User account created; verification email sent.
Main Flow	1. User enters username, email, password. 2. System validates email uniqueness and password length. 3. System generates salt, hashes password (PBKDF2, 260k iterations). 4. System inserts user record. 5. System sends verification email. 6. System redirects to login.

UC-02	Login
Actor	Registered User
Description	User submits credentials. System verifies hash and creates session.
Precondition	User is registered and email is verified.
Postcondition	Session cookie set; user redirected to main app.
Main Flow	1. User enters email/username and password. 2. System retrieves salt and hash from DB. 3. System recomputes hash and compares. 4. System creates session; sets HttpOnly cookie. 5. User sees main application.

UC-03	Search Products via Chat
Actor	Authenticated User
Description	User describes a purchase need in chat. Kart gathers intent details, issues search, and returns ranked recommendations.
Precondition	User is logged in.
Postcondition	Up to 12 ranked product cards displayed.
Main Flow	1. User sends chat message. 2. Kart asks clarifying questions (product, budget, preferences). 3. Kart issues [SEARCH: query]. 4. System calls Amazon and Walmart APIs concurrently. 5. System scores products with DistilBERT. 6. System ranks by composite score. 7. Top 12 results displayed as product cards.

UC-04	Search via Image Upload
Actor	Authenticated User
Description	Users upload a product image. System identifies the product via vision model and performs product search.
Precondition	User is logged in; fewer than 5 image searches in session.
Postcondition	Product search triggered; results displayed.
Main Flow	1. User uploads image (≤ 3 MB allowed type). 2. System validates image. 3. System encodes as Base64. 4. System sends to Groq vision model. 5. Model returns product description. 6. System runs product search on description. 7. Results displayed.

UC-05	Reset Password
Actor	Registered User
Description	User requests passwords reset, receives email link, and sets new password.
Precondition	User has a registered account.
Postcondition	Password updated; old token invalidated.
Main Flow	1. User submits email on reset form. 2. System generates secure token (valid 20 min). 3. System emails reset link. 4. User clicks link. 5. User enters new password. 6. System validates token expiry and usage. 7. System updates password hash; marks token used.

The diagram below shows all system actors (Anonymous User, Authenticated User, and external systems), the system boundary, and the relationships between actors and use cases. «includes» and «extends» dependencies are shown with dashed arrows.

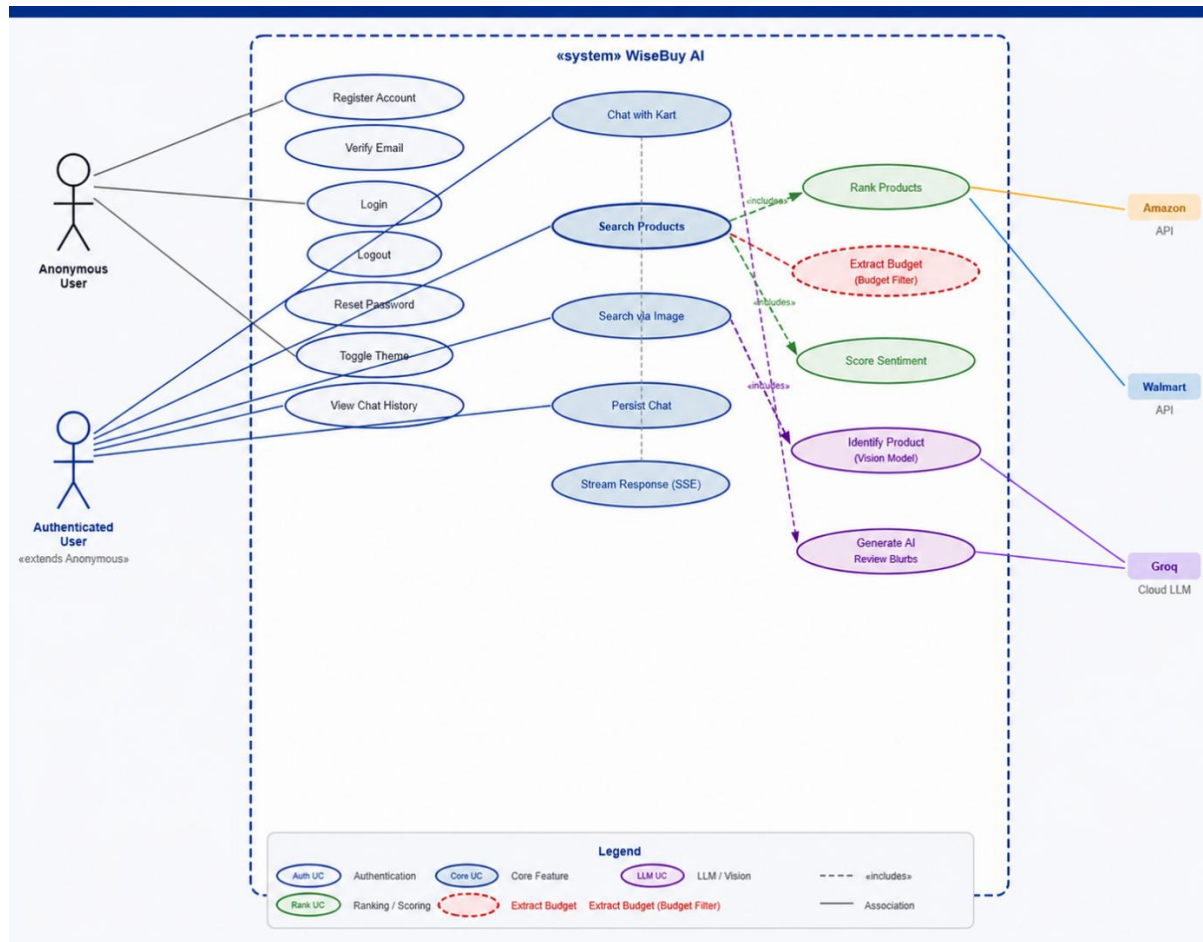


Figure 3.5.1: WiseBuy AI - Use Case Diagram showing actors, system boundaries, and functional use cases.

3.5.1 Activity Diagram

The WiseBuy AI process begins when the user sends a shopping query through the browser interface, which is received by the Flask backend using the /chat/stream endpoint. The system first performs intent analysis to determine whether enough information is available; if not, the Kart assistant asks clarifying questions and streams responses in real time using Server-Sent Events (SSE). Once the intent is complete, the backend extracts the search query and budget details, then launches parallel API requests to Amazon and Walmart for live product retrieval. If the query includes an image, the image is encoded and processed using the Groq Vision model (llama-4-scout-17b) to generate a product search query automatically.

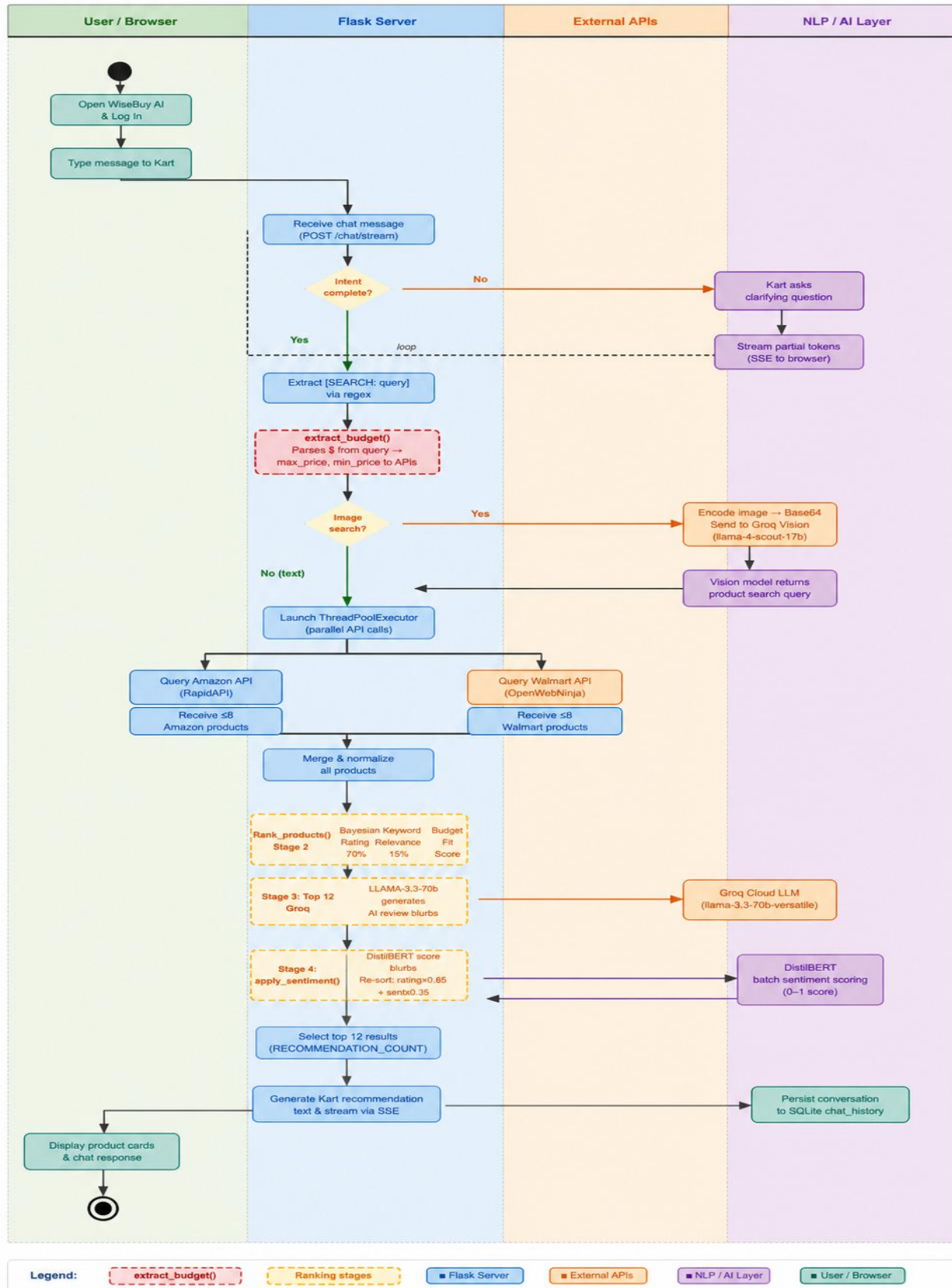


Figure 3.5.2: WiseBuy AI - Activity Diagram showing the end-to-end product search workflow

IV. SYSTEM DESIGN

4.1. Architectural Design

WiseBuy AI is developed using layered monolithic architecture. In this approach, the complete application is built and deployed as one system, but the responsibilities are divided into separate logical layers. This structure was suitable for the project because WiseBuy AI includes different connected features such as user login, chatbot interaction, product search, image search, product ranking, saved chat history, and account management.

The system's front end serves as the presentation layer. The main interface, which allows visitors to log in, browse as a guest, interact with the product advisor, add images, and read product recommendations, is built with HTML, CSS, and JavaScript.

The frontend and backend communicate using API requests.

Flask is used to manage the backend. It takes requests from the frontend, handles user activities, maintains the primary application routes, and replies to the interface. This involves managing image-based searches, processing text-based product searches, setting up chats, processing login and signup, and storing messages.

The system's primary decision-making component is located in the business logic layer. It manages the chatbot dialogue, creates search queries, assesses merchandise, examines uploaded photos, and enhances the system also connects with AI services to understand user requirements and generate helpful product suggestions.

For storing data, the project uses an SQLite database. The database keeps records of users, chats, messages, passwords, reset details, and saves product results. This allows registered users to return to their previous conversations and recommendations.

The application also depends on external services. Product information is collected from Amazon and Walmart APIs, while AI-based features are supported through Groq and a local sentiment analysis model.

These services enable the system to generate more relevant and meaningful product recommendations.

Overall, this architecture simplifies project management while clearly separating the system's key components. As the program is not too huge, a layered monolithic structure is feasible, easy to test, and simple to deploy.

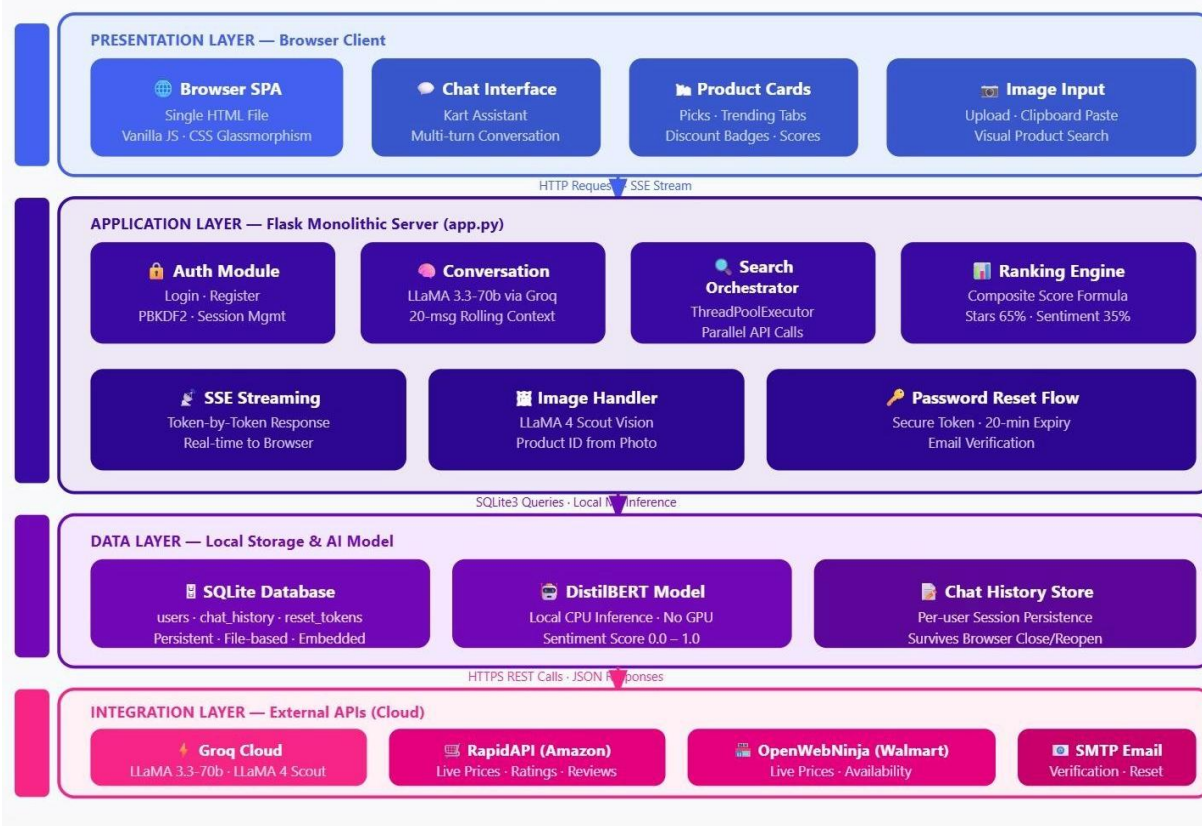


Figure 4.1 WiseBuy -A layered Monolithic Architecture

Layer 1: Presentation shows everything the user sees in the browser - the chat interface, product cards, and image input.

Layer 2: Application is the core Flask monolith - all business logic lives here in one process: authentication, conversation management, search orchestration, ranking, streaming, and image handling.

Layer 3: Data shows what stays local on the server - SQLite for persistence, DistilBERT for sentiment scoring (no external call), and chat history storage.

Layer 4: Integration shows every external cloud service Flask call outward - Groq Cloud for LLMs, RapidAPI for Amazon, OpenWebNinja for Walmart, and SMTP for email.

4.2. Development Model

WiseBuy AI was developed using an Agile-inspired iterative and incremental strategy. The project wasn't finished fully at once. Instead, it was created incrementally, with each feature introduced, tested, and improved before going on to the next section.

At first, the basic web application and user interface were developed. This provided a basis for the project and allowed users to interact with the system. Following that, the chatbot feature was added, allowing customers to describe their desired purchases and receive recommendations from the system.

In subsequent stages, product search features were introduced by linking the program to Amazon and Walmart APIs. Once the basic search flow was operational, further complex features were added, including user authentication, saved conversation history, image-based product search, product ranking, sentiment analysis, password reset, and account deletion.

This development technique was ideal for architecture since it allowed each new feature to be added to its associated layer without requiring a complete system rebuild. For example, the frontend interface was improved, search logic was introduced to the backend, and user data was stored in the database layer.

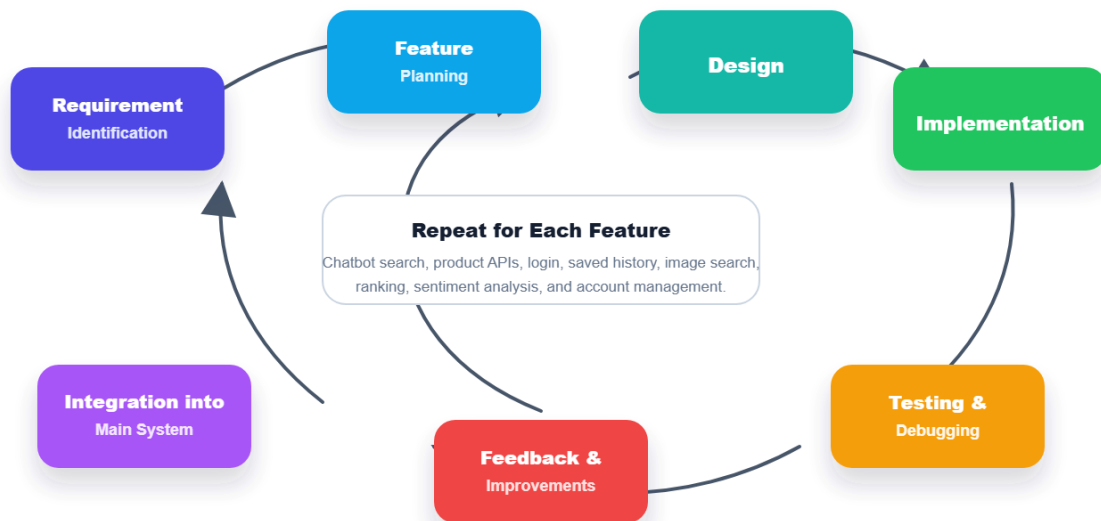


Figure 4.2 WiseBuy AI- Agile Iterative and Incremental Development Cycle

The overall development cycle included determining requirements, planning a feature, building it, testing it, addressing issues, and finally integrating it into the main system. This helped the project improve gradually and made it easier to manage errors and changes during development.

V. IMPLEMENTATION

5.1 Technology Stack

The architecture of WiseBuy AI is documented using the 4+1 view model, each view capturing a distinct perspective of the system for a different set of stakeholders. The five views are: Logical View (system structure and responsibilities), Physical View (hardware and deployment topology), Development View (code organization and module dependencies), Process View (runtime interaction flows) and Use Case View (cross-cutting scenarios already described in the SRS). Each view is supported by a formal UML diagram.

5.1.1 Logical View (Component Diagram)

The logical view describes the system in terms of functional components and their interfaces.

WiseBuy AI is partitioned into three major subsystems: the Browser / SPA client, the Flask Application Server, and the External Services layer. Figure 5.1 presents the Component Diagram illustrating these partitions and their communication interfaces.

Browser / SPA Client

The client subsystem consists of four JavaScript modules served as static assets by Flask. `index.html` is the single HTML entry point rendered for all routes; routing is handled client-side. `chat.js` manages the SSE Event Source connection, token streaming display, and product card rendering. `Auth.js` implements login, registration, and password-reset form handling. `products.js` formats ranked product results as interactive cards. `theme.js` toggles between light and dark glass morphism themes.

Flask Application Server

The server subsystem is organized as a set of route controllers backed by service modules. The Auth Controller handles `/register`, `/login`, `/logout`, and `password-reset` routes; it delegates credential hashing and session management to the Session Manager, which implements PBKDF2-HMAC-SHA256 with 260,000 iterations. The Chat Controller exposes the `/chat/stream` SSE endpoint; each request invokes the Chatbot Module (`chatbot.py`), which submits conversational turns to the Groq LLM and monitors its response for the `[SEARCH: ...]` directive. When found, the Chat Controller hands off to the Search Orchestrator, which concurrently calls both product scrapers. The Ranking

Engine then scores result via the Sentiment Analyzer and applies the composite formula before returning to the top 12 products.

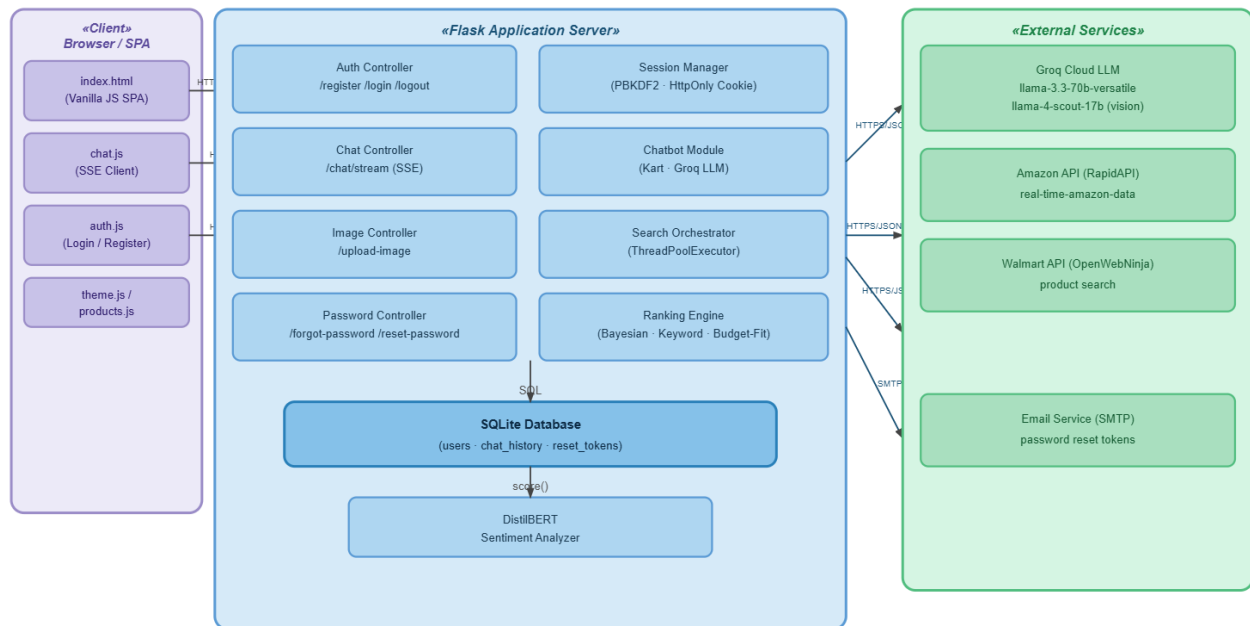


Figure 5.1.1 WiseBuy AI- Component Diagram (Logical View)

The Ranking Algorithm Used

WiseBuy AI uses a **Composite Weighted Scoring Algorithm** - a custom formula combining three signals:

Step 1 - Raw Score:

$$\text{raw_score} = (\text{star_rating} \times 100) + (\text{review_count} \times 0.01) + (\text{sentiment_score} \times 5)$$

- $\text{star_rating} \times 100$ scales the 0–5-star rating to a 0–500 range
- $\text{review_count} \times 0.01$ adds a small volume bonus (more reviews = slightly more trust)
- $\text{sentiment_score} \times 5$ boosts products with strongly positive review language

Step 2 - Final Composite Score:

$$\text{final_score} = (\text{star_score} \times 0.65) + (\text{sentiment_score} \times 0.35)$$

- 65% weight on normalized star score
- 35% weight on DistilBERT sentiment score

The Algorithm states that star ratings alone are unreliable - a 4.9-star product with 5 reviews and a 4.7-star product with 8,000 reviews are not equally trustworthy. Adding Distil BERT sentiment

(which reads the actual text of reviews) corrects this bias. The 65/35 split was chosen during testing to balance objective rating data with qualitative review language.

5.1.2 Physical View (Deployment Diagram)

WiseBuy AI follows a client-server deployment architecture where users access the system through a modern web browser on desktop or mobile devices. The frontend operates as a Vanilla JavaScript Single-Page Application (SPA) with SSE EventSource support for real-time chat streaming over HTTPS.

The backend runs on an Ubuntu Linux server using Python 3.11 and Flask 3.x. The Flask WSGI application (app.py) acts as the central controller and communicates with the chatbot module, scraper layer, ranking engine, DistilBERT sentiment model, SQLite database, and frontend assets stored locally on the server.

The chatbot layer uses Groq Cloud APIs through HTTPS REST communication. The llama-3.3-70b-versatile model handles conversational chat completions, while llama-4-scout-17b processes image-based product identification and vision search tasks.

The scraper layer connects to external marketplace services including Amazon Product API through RapidAPI and Walmart Search API through OpenWebNinja. Product results retrieved from these services are processed by the ranking engine using scoring, normalization, and sentiment analysis before recommendations are streamed back to the client.

The DistilBERT sentiment model is hosted locally for CPU inference and loads dynamically on first use. User sessions, chat history, and authentication data are stored inside the local SQLite database (wisebuy.db). The system also integrates with an SMTP email server for password reset functionality and account recovery operations.

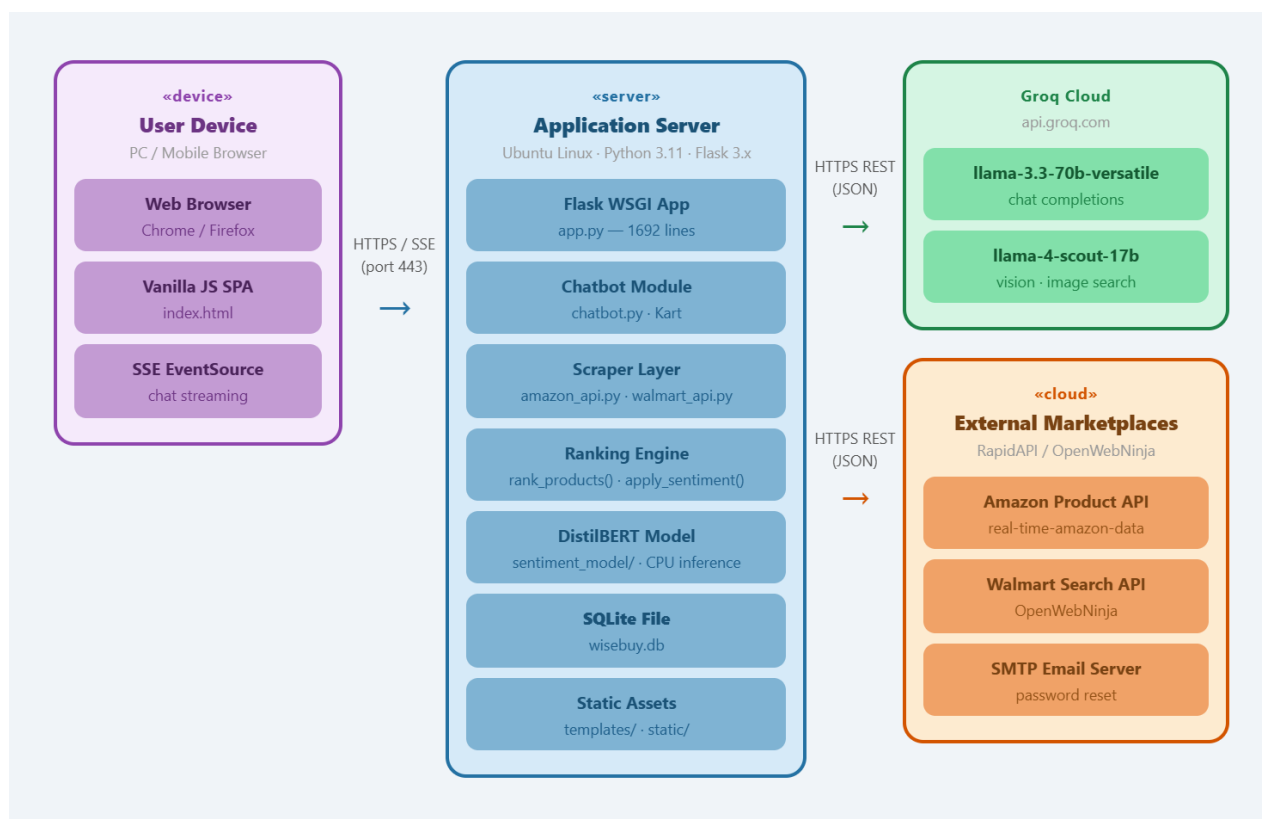


Figure 5.1.2 WiseBuy AI Deployment Diagram (Physical View)

5.1.3 Development View (Package Diagram)

The WiseBuy AI project is structured into modular packages with clearly separated responsibilities to improve maintainability and scalability. The root projectDemo/ directory contains the core application folders including app/, chatbot/, scraper/, templates/, static/, database/, and ranking_engine/, along with the SQLite database file and project dependencies.

The app/ package contains the main Flask backend application. The app.py module manages Flask routes, SSE streaming, authentication sessions using PBKDF2, image upload handling, and ThreadPoolExecutor-based parallel product search execution.

The chatbot/ package contains chatbot.py, which manages the Kart AI assistant, Groq-powered conversation handling, [SEARCH:] directive detection, and rolling 20-message context management for conversational continuity.

The `scraper/` package handles external product retrieval and sentiment analysis. The `amazon_api.py` and `walmart_api.py` modules connect to RapidAPI and OpenWebNinja services, while `sentiment_analyzer.py` performs DistilBERT batch sentiment scoring through the `score_batch()` function.

The frontend layer is divided into `templates/` and `static/` directories. The `templates/` folder stores Jinja2 HTML pages including authentication pages, while the `static/` folder contains glassmorphism-based CSS styling and JavaScript modules such as `chat.js`, `auth.js`, and `products.js`. The `database/` package uses the SQLite database file `wisebuy.db` to store users, chat history, and password reset tokens through direct `sqlite3` access without an ORM layer.

The `ranking_engine/` package performs recommendation orchestration and ranking operations. Modules such as `ranker.py`, `feature_builder.py`, `scorer.py`, `model.py`, and `rules.py` handle feature extraction, weighted scoring, learning-to-rank logic, business-rule boosting, and recommendation normalization using signals like ratings, keyword relevance, sentiment scores, and budget fit.

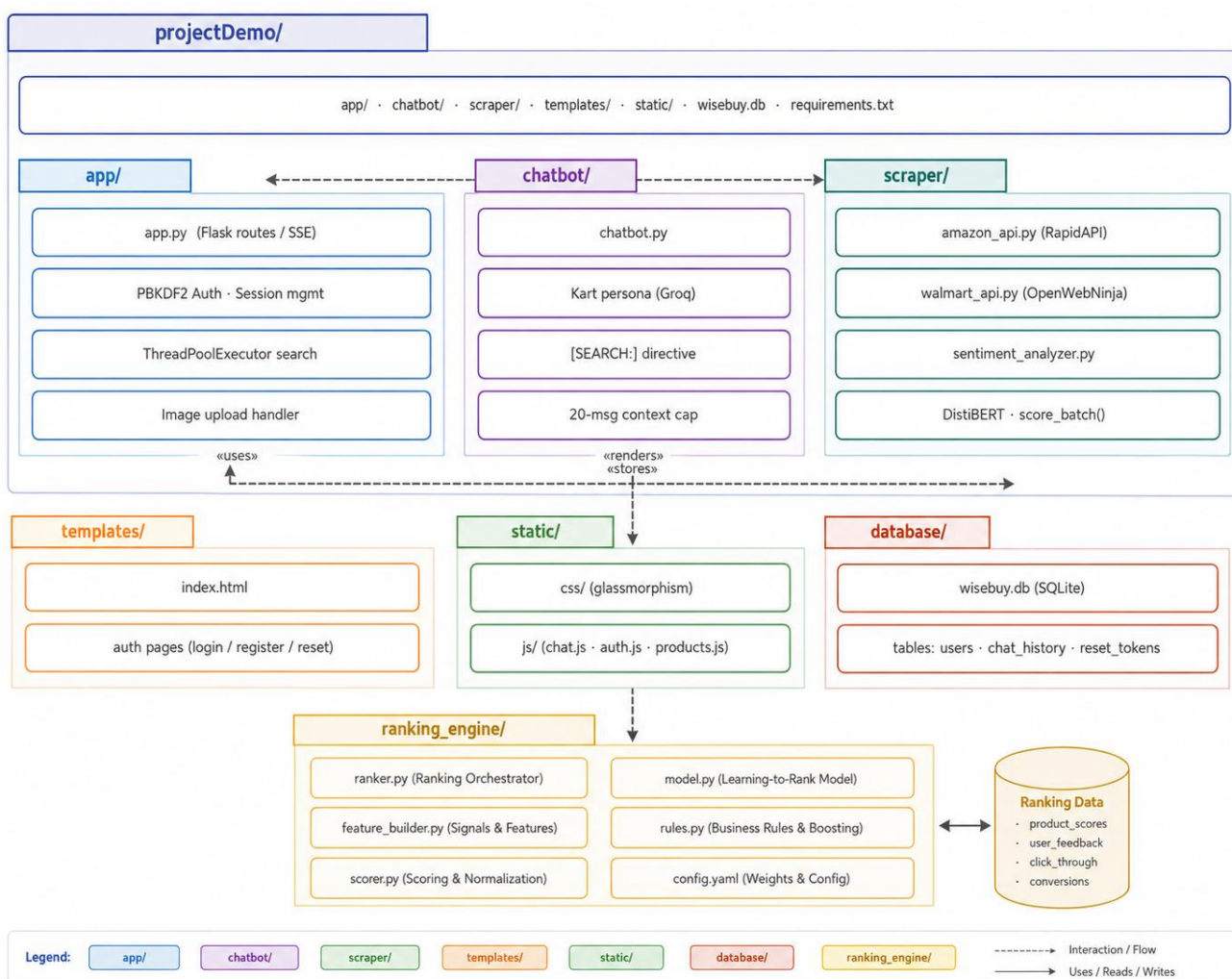


Figure 5.1.3 WiseBuy AI Package Diagram (Development View)

5.1.4 Process View (Sequence Diagram)

When a user submits a shopping query through the Browser PWA, the frontend sends a POST /chat/stream request to the Flask Server using an SSE connection for real-time response streaming. The Flask server forwards the user message and recent conversation history to the Kart chatbot module through the `chat_with_kart(message, history)` function.

The Kart AI assistant processes the request using the Groq LLM and generates either a clarification response or a [SEARCH: query] directive when purchase intent is identified. The Flask server extracts the search query and optional budget values using regular expression parsing before passing the request to the Search Orchestrator.

The Search Orchestrator launches concurrent API requests using a `ThreadPoolExecutor` to both Amazon and Walmart product APIs. Product results retrieved from RapidAPI and OpenWebNinja are merged into a unified product collection.

The merged product reviews are then forwarded to the DistilBERT sentiment module using the `score_batch(review_texts)` function, which returns sentiment scores ranging from 0.0 to 1.0 for all retrieved products in a single batch inference pass.

Next, the Ranking Engine receives the product list, query information, budget constraints, and sentiment scores. The ranking pipeline performs feature extraction, weighted scoring, normalization, and business-rule boosting using factors such as ratings, price relevance, sentiment confidence, and budget fit. The final composite scoring formula is calculated as:

$$score = rating \times 100 + reviews \times 0.01 + sentiment \times 5$$

After ranking, the top 12 products are selected and converted into recommendation responses by the Kart assistant. The Flask server streams the recommendation text and product cards back to the browser through SSE events in real time. Finally, the full user and assistant conversation turn is stored inside the SQLite `wisebuy.db` database while maintaining a rolling 20-message conversation history window.

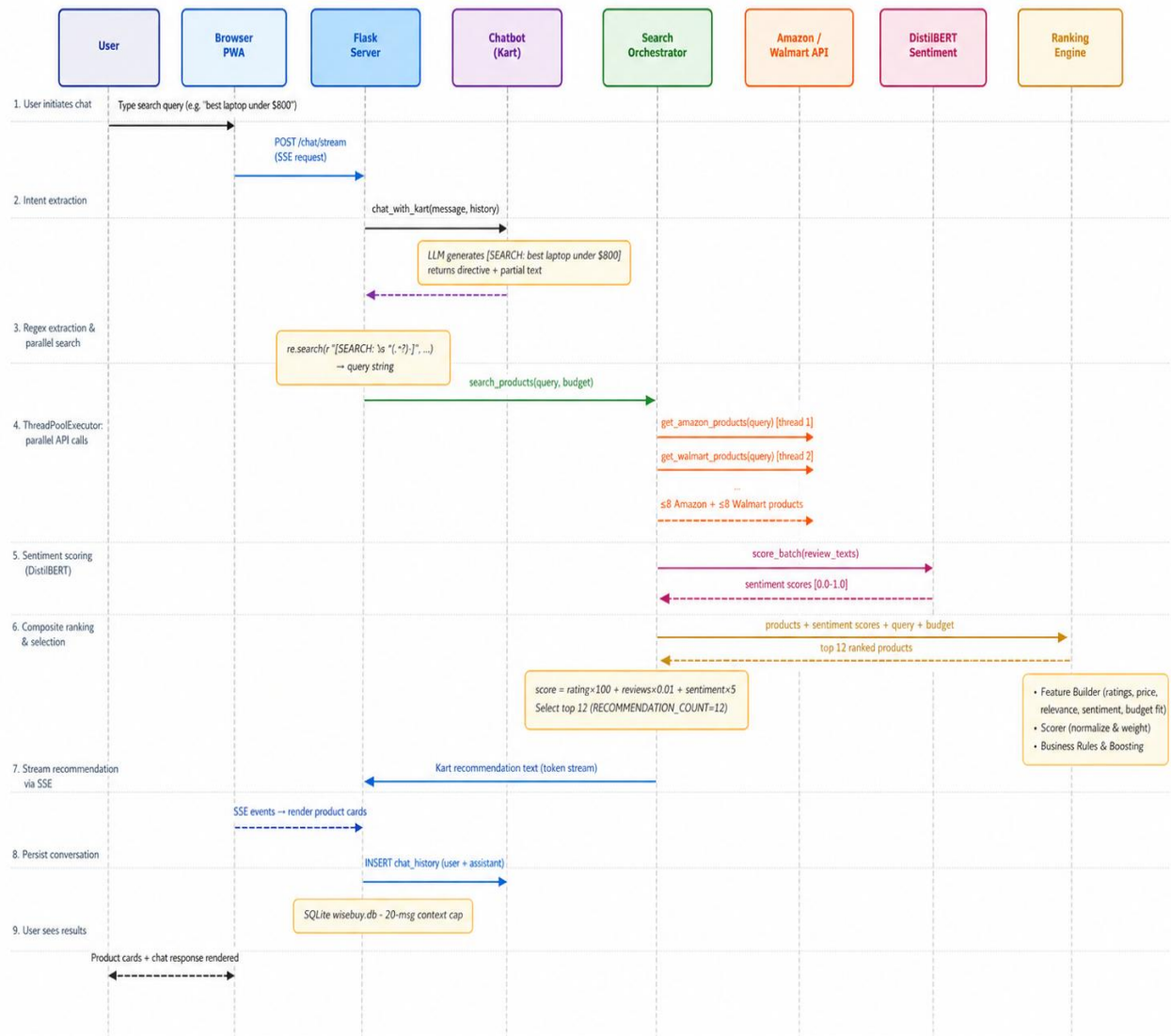


Figure 5.1.4: WiseBuy AI Sequence Diagram (Process View - Product Search Flow)

5.2 Database Design

WiseBuy AI uses SQLite 3 as its database, with the file wisebuy.db created automatically on first launch through the `init_db()` function in `app.py`. The schema contains three tables - `users`, `chat_history`, and `password_reset_tokens` each serving a distinct purpose: authentication, conversation persistence, and secure password recovery. The relationship between them is straightforward: one user owns zero or more chat history records and zero or more password reset tokens, with no many-to-many relationships involved. Foreign key constraints are intentionally omitted from the SQLite schema and enforced at the application layer instead, keeping the database portable and lightweight across different deployment environments.

launch through the `init_db()` function in `app.py`. The schema contains three tables - `users`, `chat_history`, and `password_reset_tokens` each serving a distinct purpose: authentication, conversation persistence, and secure password recovery. The relationship between them is

straightforward: one user owns zero or more chat history records and zero or more password reset tokens, with no many-to-many relationships involved. Foreign key constraints are intentionally omitted from the SQLite schema and enforced at the application layer instead, keeping the database portable and lightweight across different deployment environments.

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique user identifier
username	TEXT	NOT NULL	Display name chosen at registration
email	TEXT	NOT NULL UNIQUE	Primary login credential; must be unique across all users
password_hash	TEXT	NOT NULL	PBKDF2-HMAC-SHA256 hash (260,000 iterations, 32-byte salt, hex-encoded)
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Account creation timestamp (UTC)

Table 5.2.1 users

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique record identifier
user_id	INTEGER	NOT NULL	References users.id (application-enforced)
role	TEXT	NOT NULL	“user” or “assistant” identifies message originator
content	TEXT	NOT NULL	Full message text; for assistant messages includes product summary JSON
timestamp	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Message creation time (UTC); used for history ordering and 20-msg cap enforcement

Table 5.2.2 chat_history Table

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique token record identifier
user_id	INTEGER	NOT NULL	References users.id
token	TEXT	NOT NULL UNIQUE	Cryptographically random 32-byte token (hex-encoded, 64 chars)
expiry	TIMESTAMP	NOT NULL	Expiry time = created_at + 20 minutes; validated before accepting reset
used	INTEGER	DEFAULT 0	0 = active, 1 = consumed; one-time-use enforcement

Table 5.2.3 password_reset_tokens Table

5.3. Security Design

WiseBuy AI uses a glassmorphism design language throughout - frosted-glass translucent panels with soft shadows and blue accent colors set against a deep navy gradient background, fully responsive from 320px mobile to 4K desktop. The authentication screen is the entry point, presenting a centered glass card with tabbed Login and Register forms, inline validation, and a password reset flow that generates a 32-byte token stored with a 20-minute expiry and delivered by email. Once authenticated, users land on the main chat screen split into two panels - the left chat panel occupying 45% of the screen width renders alternating message bubbles with real-time SSE token streaming and a typing indicator, while the right product results panel occupying 55% displays up to 12 ranked product cards in a scrollable grid. Each product card shows a thumbnail, title, star rating, review count, a color-coded sentiment bar, price in large blue text, a store badge identifying Amazon or Walmart, and a View button that opens the product page directly in a new tab. The top three ranked products are marked with gold, silver, and bronze medal badges, and cards are sorted by composite score descending. A History screen accessible from the navigation bar retrieves the user's full conversation log from `chat_history` ordered by timestamp and presents each session as an expandable accordion item.

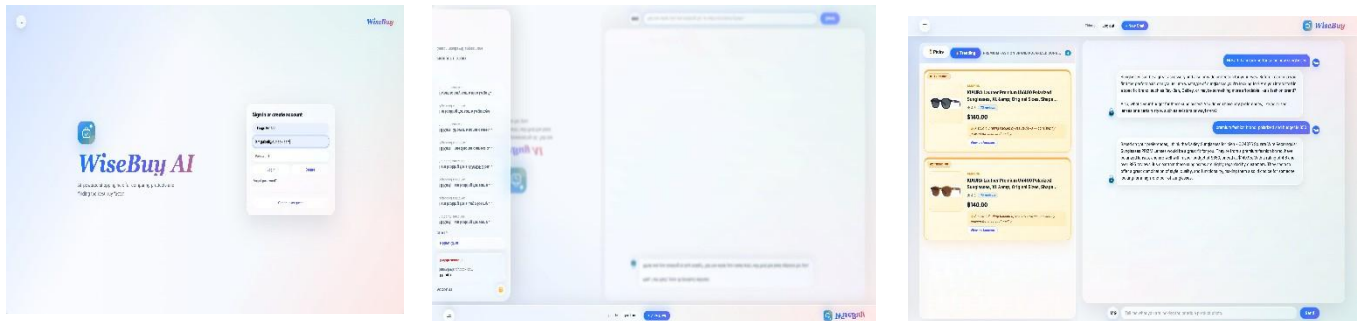


Figure 5.3.1: User Interface

5.4. Feature-by-Feature Walkthrough

Feature 1 - Sign Up, Login, and Guest Mode Users can register with an email and password, log in to an existing account, or continue as a guest without signing up. Passwords are secured using PBKDF2-SHA256 hashing and must be at least 8 characters containing one letter and one digit.

Feature 2 - Chat to Find Products Users type what they are looking for and Kart asks follow-up questions about budget and features before searching Amazon and Walmart simultaneously.

Results are ranked using a composite score combining star rating, keyword match, budget fit, and DistilBERT sentiment analysis.

Feature 3 - Image Search (Upload or Paste a Photo) Users can upload a product photo or paste one directly from the clipboard using Ctrl+V, and the vision model identifies the product and generates up to five search query variations. All queries run simultaneously against both platforms and results are ranked by how closely they match the image.

Feature 4 - Sentiment Scoring DistilBERT reads product review text and assigns a sentiment score between 0.0 and 1.0, displayed as a colored bar on every product card - green for positive, amber for mixed, and red for complaints. The final ranking combines star rating at 65% weight and sentiment score at 35%, meaning review quality directly influences which product ranks highest.

Feature 5 - Picks and Trending Tabs The top 4 highest-scoring products appear in the Picks tab with a Best Match badge on the number one card, while the 2 most-reviewed products from the remaining results appear as gold hero cards in the Trending tab. This split helps users choose between the AI's best recommendation and what the wider buying public has trusted most.

Feature 6 - Discount Badges Every product card displays the current price alongside the original price with a strikethrough and a percentage-off badge calculated automatically from both values. This gives users an immediate sense of the deal without needing to visit the retailer's website first.

Feature 7 - Password Reset by Email Users who forget their password can request a reset link from the login screen, which triggers a secure 192-bit token sent to their registered email address. The link expires after 20 minutes and can only be used once, after which the token is permanently invalidated.

Feature 8 - View Product on Amazon or Walmart Each product card stores the direct URL to its listing on Amazon or Walmart, and clicking the View button opens that exact product page in a new browser tab. WiseBuy AI handles no payments - all purchasing happens directly and securely on the retailer's official website.

Feature 9 - Product Brief in Chat Interface After every search, Kart writes a natural-language explanation in the chat bubble identifying the best match and explaining why it ranked first - covering rating, review volume, price fit, and key features. This summary streams word by word in real time so users understand the recommendation rather than just seeing a list of products.

Feature 10 - Dark and Light Theme A toggle button in the top navigation bar switches the entire interface between dark mode - a deep purple gradient with frosted glass panels - and light mode - a clean white layout suited for bright environments. The selected theme applies instantly without any page reload and remains active for the duration of the session.

VI. TESTS AND RESULTS

All features of WiseBuy AI were tested manually between April 28 and May 3, 2026. The table below shows each test case with its result.

Component	Details
Operating System	Windows 11 (development) / Ubuntu 22.04 (server target)
Python Version	3.11.9
Browsers Tested	Google Chrome 124, Mozilla Firefox 125
Internet Connection	Standard home broadband (~100 Mbps)
Groq API Tier	Free tier - limited to 30 requests/minute

Table 6.1 Test Environment

#	What Was Tested	How It Was Tested	Result
TC-01	New account registration	Valid email + Password with letter and digit	PASS
TC-02	Duplicate email rejected	Register twice with same email	PASS
TC-03	Weak PASSWORD rejected	PASSWORD with no numbers ("abcdefgh")	PASS
TC-04	Login works	Correct email and PASSWORD	PASS
TC-05	Wrong PASSWORD blocked	Correct email, wrong PASSWORD	PASS
TC-06	Guest mode works	Click "Continue as Guest"	PASS
TC-07	Chat search works	"Best laptop under \$800"	PASS - 8 products returned and ranked
TC-08	It asks for details	Type "blue" (too vague)	PASS – It asked clarifying questions
TC-09	Empty message blocked	Click Send with no text	PASS - UI prevents it
TC-10	Photo upload search	JPEG of wireless earbuds via file picker	PASS - vision model identified product

TC-11	Clipboard paste search	CTRL +V image from another tab	PASS - paste detected and processed
TC-12	Large file blocked	Image > 3 MB uploaded	PASS - error shown
TC-13	Picks tab correct	Search returning 8 results	PASS - top 4 shown, #1 has Best Match badge
TC-14	Trending tab correct	Search returning 8 results	PASS - 2 most-reviewed shown as gold cards
TC-15	Discount badge shows	Product with original price > current price	PASS - -XX% badge rendered
TC-16	Sentiment bar shows	Any product search	PASS - colored bar on every card
TC-17	Password reset email	Registered email + SMTP configured	PASS - email received, valid 20 min
TC-18	Unknown email handled	Reset request for unregistered email	PASS - generic message (no info leaked)
TC-19	Local dev token	Reset on localhost, no SMTP	PASS - token returned in API response
TC-20	Expired token rejected	Use reset link after 20 min	PASS - error shown
TC-21	Streaming responses	Chat on slow connection	PASS - words appear incrementally
TC-22	Multiple users at once	5 simultaneous chat sessions	PASS - all responded correctly

Table 6.2 Test Results

Result: 22 out of 22 test cases Passed. All core features work as expected.

6.1 Demo Scenarios: (Step-by-Step Examples)

Scenario A - Registered User Shopping for a Gift

1. User registers a new account and logs in.
2. Types: "I'm looking for a birthday gift for a 10-year-old who loves LEGO, budget \$40."
3. It asks one clarifying question about LEGO theme preference, then searches.
4. 8 products fetched and ranked. Picks tab shows top 4, Trending shows 2 most-reviewed sets.

5. User spots a -15% discount badge on a Picks card and clicks "View on Amazon."

Scenario B - Clipboard Paste Image Search

1. User sees a backpack photo on another website and copies it (Ctrl+C).
2. Switches to WiseBuy AI and presses Ctrl+V anywhere on the page.
3. The image is automatically captured and sent to the vision AI.
4. Vision model identifies the product and creates 4 search query variations.
5. All queries run at the same time against Amazon and Walmart. Results ranked by image match score.

6.2 Limitations

1. Groq free API: max 30 requests/min - busy times may fail.
2. SQLite can't handle many users at once writing data.
3. Sentiment AI is slow (~100ms per product on CPU).
4. No price history.
5. Trending tab shows exactly 2 - can be sparse

VIII. FUTURE ENHANCEMENTS

In the future, WiseBuy AI can be enhanced with more advanced features that can scale the system, making it more accurate and user-friendly. Some possible enhancements are:

1. Support for More E-Commerce Platforms

Currently, the system searches for products on Amazon and Walmart. We plan to integrate it with more platforms such as eBay, Best Buy, Target, Flipkart, or local shopping websites that can be added. This will give users more product options and a better pool of prices.

2. Advanced Price Comparison

The system can be enhanced to search for and compare the same or similar products across multiple

platforms and highlight the best deal based on price, rating, delivery cost, and Availability.

3. Price Tracking and Alerts

The system can be added with a price tracking feature where users can save a product and receive real-time alerts when a price drops. This will enable users to make faster decisions and take advantage of a product deal.

4. Personalized Recommendations

The system can learn from user behavior by analyzing preferences, previous searches, saved chats, and selected products to provide more personalized product suggestions.

5. Improved Image Search Accuracy

The image search feature can be improved by using more advanced computer vision models. This would help the system identify product type, brand, model number, color, and other visible details more accurately.

6. Voice-Based Search

Instead of typing all search queries, voice input can be added so users can search for products by speaking. This would make the application more accessible and convenient.

7. Product Comparison Table

A comparison table feature can be added to help users with more side-by-side information to compare selected products based on price, rating, reviews, features, platform, and sentiment score.

8. Review Summarization

The system can summarize real customer reviews into short pros and cons. This would help users understand product quality without reading many reviews manually.

9. Mobile App Version

A mobile application can be developed for Android and iOS so users can access WiseBuy AI more easily from their smartphones.

10. Advanced Security Features

Future versions can include email verification, two-factor authentication, stronger password policies, and improved session management to make user accounts more secure.

11. Cloud Deployment

The project can be deployed on cloud platforms such as AWS, Azure, Google Cloud, or Render, streamlining the process to be accessible from reliable hosts.

12. Scalable Database Upgrade

The SQLite database can be replaced with PostgreSQL or MySQL offering better performance and scalability when the number of users and saved chats increases.

13. Admin Dashboard

An admin dashboard can be added to monitor users, product searches, API usage, error logs, and

overall system performance.

14. Recommendation Accuracy Improvement

The system's ranking algorithm can be improved by expanding and considering more factors such as delivery time, return, policy, seller reliability, discount percentage, and verified reviews.

VII. CONCLUSION

WiseBuy AI is a conversational shopping assistant that helps users find and compare products from Amazon and Walmart without leaving a single interface. Instead of asking someone to open multiple browser tabs, read through sponsored listings, and manually weigh prices against reviews, WiseBuy AI handles that process through a back-and-forth conversation. A user types what they are looking for a laptop under a certain budget, a backpack that fits carry-on requirements, a gift for a specific occasion - and the system queries both marketplaces in parallel, scores the results using real review sentiment, and returns a ranked comparison in plain language. The user can refine, ask follow-up questions, or upload a photo of a product they already saw somewhere and ask the system to find it cheaper. That entire flow happens inside one chat window.

The shift from traditional online shopping is meaningful. In the conventional model, the user does most of the work. They pick a platform, form a search query, sort through results that mix advertisements with organic listings, open individual product pages to read reviews, and then repeat the same process on a second platform if they want to compare prices. The cognitive load is real, and it falls entirely on the shopper. WiseBuy AI moves that burden to the system. The conversation guides users toward giving enough context - budget, purpose, preferred brands, concerns about quality - that the search query it generates is more precise than what most people would type into a search box themselves. A user who types "good headphones" into Google gets ten million results. A user who tells WiseBuy AI they want noise-cancelling headphones under \$80 for commuting and have had bad experiences with ear fatigue gets a filtered, ranked shortlist with sentiment-backed reasoning attached.

The system also makes product information more accessible. Review scores are recomputed using DistilBERT sentiment analysis on actual customer feedback rather than relying solely on the star rating that sellers can influence through promotional campaigns. Trending products are identified from live marketplace data rather than curated editorial lists. The chat history persists across sessions, so users can return to earlier searches without starting over. These features collectively

lower the barrier to making an informed purchase decision, particularly for users who find traditional e-commerce interfaces overwhelming or time-consuming.

WiseBuy AI does not replace the marketplace. It acts as a layer between the user and the marketplace - one that understands natural language, remembers context, and does the comparison work automatically. For everyday shoppers, that changes the experience from research into conversation. The goal was never to build the most technically sophisticated system. It was to prove that a well-designed conversational layer could make product discoveries faster, more relevant, and less frustrating than the current default. Based on what the prototype delivers, that goal was met.

REFERENCES

- [1] Y. Mullagura, "Product recommendation system using sentiment analysis on e-commerce application," B.Sc. thesis, Faculty of Computing, Blekinge Institute of Technology, Karlskrona, Sweden, 2023.
- [2] P. Tangarajan, A. A. Rajasekar, M. Rathi, V. R. Dandin, and O. Ersoy, "Contextually aware e-commerce product question answering using RAG," *arXiv preprint arXiv:2508.01990*, 2025.
- [3] [Online]. Available: <https://arxiv.org/abs/2508.01990>
- [4] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," *arXiv preprint arXiv:1910.01108*, 2019. [Online]. Available: <https://arxiv.org/abs/1910.01108>
- [5] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei et al., "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023. [Online]. Available: <https://arxiv.org/abs/2307.09288>
- [6] Groq API Documentation. <https://console.groq.com/docs>
- [7] Flask Documentation v3.x. <https://flask.palletsprojects.com/>
- [8] Hugging Face - DistilBERT Documentation.
<https://huggingface.co/distilbert-base-uncased-finetuned-sst-2-english>
- [9] IEEE Std 830-1998, for Writing